



**CEBU INSTITUTE OF TECHNOLOGY**  
**U N I V E R S I T Y**

# IT342-Section SYSTEMS INTEGRATION AND ARCHITECTURE 1

---

## FUNCTIONAL REQUIREMENTS SPECIFICATION (FRS)

---

Project Title: Mini App – User Registration & Authentication

Prepared By: Galo, Margel Destine Krizia

Date of Submission: February 16, 2026

Version: v2

# Table of Contents

- 1. Introduction.....3
  - 1.1. Purpose..... 3
  - 1.2. Scope..... 3
  - 1.3. Definitions, Acronyms, and Abbreviations..... 3
- 2. Overall Description.....3
  - 2.1. System Perspective..... 3
  - 2.2. User Classes and Characteristics.....3
  - 2.3. Operating Environment..... 3
  - 2.4. Assumptions and Dependencies..... 3
- 3. System Features and Functional Requirements.....3
  - 3.1. Feature 1:.....3
  - 3.2. Feature 2:.....3
- 4. Non-Functional Requirements..... 3
- 5. System Models (Diagrams)..... 4
  - 5.1. ERD..... 4
  - 5.2. Use Case Diagram..... 4
  - 5.3. Activity Diagram.....4
  - 5.4. Class Diagram.....4
  - 5.5. Sequence Diagram.....4
- 6. Appendices.....4

## 1. Introduction

### 1.1. Purpose

The purpose of this system is to provide a secure, centralized authentication service that manages user identity across a React web application and a Kotlin mobile application. The intended audience includes project evaluators and system maintainers.

### 1.2. Scope

The system handles user registration, login, and profile management. It includes a Spring Boot backend, an H2 database for persistence, a React frontend, and an Android mobile app. Its boundaries are limited to local network authentication and does not include external OAuth providers.

### 1.3. Definitions, Acronyms, and Abbreviations

**JWT (JSON Web Token):** A stateless token used to securely transmit user identity between client and server.

**H2:** An in-memory relational database used for storing user credentials during development.

**API (Application Programming Interface):** The set of endpoints (e.g., /api/auth/login) used by the apps to communicate with the server.

## 2. Overall Description

### 2.1. System Perspective

The system operates as a client-server architecture where a single Spring Boot backend acts as the source of truth for two distinct client platforms: Web (React) and Mobile (Kotlin).

### 2.2. User Classes and Characteristics

**Guest User:** Can access the landing, login, and registration screens.

**Authenticated User:** Can access the Dashboard and Profile screens after successfully providing valid credentials.

### 2.3. Operating Environment

**Backend:** Spring Boot 3.x, Spring Security, H2 Database.

**Web:** React with react-router-dom.

**Mobile:** Android Studio, Kotlin, Retrofit for API calls.

## 2.4. Assumptions and Dependencies

**Network:** The mobile emulator must use 10.0.2.2 to communicate with the local Spring Boot server.

**Statelessness:** The system assumes JWTs are stored client-side (Local Storage or SharedPreferences) to maintain sessions.

## 3. System Features and Functional Requirements

Describe each major feature of the system and its functional requirements.

### 3.1. Feature 1: User Authentication (Login/Register)

**Description:** Allows new users to create accounts and existing users to securely enter the system.

#### Functional Requirements:

- The system shall validate that emails and usernames are unique before saving to the H2 database.
- The system shall encrypt passwords using BCryptPasswordEncoder before storage.
- The system shall return a JWT accessToken upon successful login.

### 3.2. Feature 2: Protected User Profile

**Description:** A secure area where authenticated users can view their account details.

#### Functional Requirements:

- The system shall restrict access to /api/user/me to requests containing a valid "Bearer" token.
- The system shall retrieve the current user's details (ID, Email, Name) from the SecurityContextHolder.
- The mobile and web clients shall automatically redirect to the landing page if the token is invalid or missing.

## 4. Non-Functional Requirements

**Security:** All sensitive endpoints must be protected.

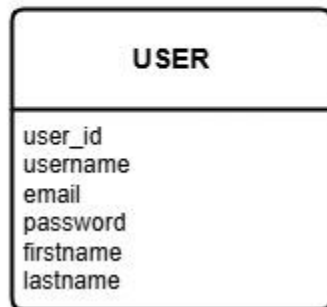
**Usability:** The UI must maintain consistency across platforms, utilizing rounded CardViews and modal confirmation dialogs for logout.

**Performance:** The H2 console must remain accessible for real-time data debugging without blocking frame options.

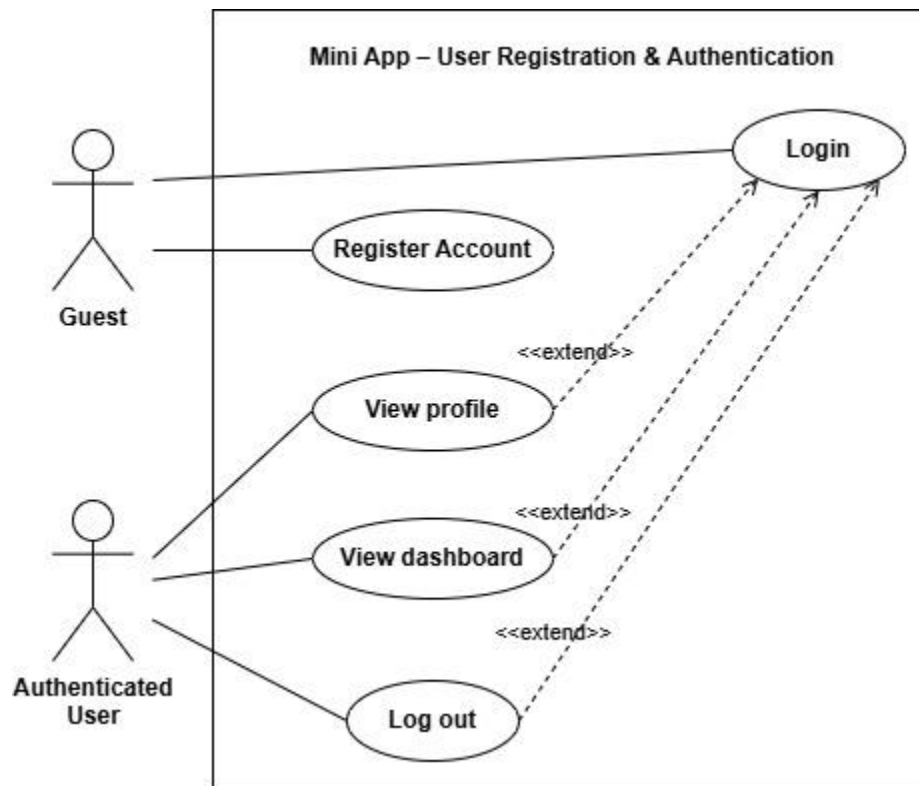
## 5. System Models (Diagrams)

### 5.1. ERD

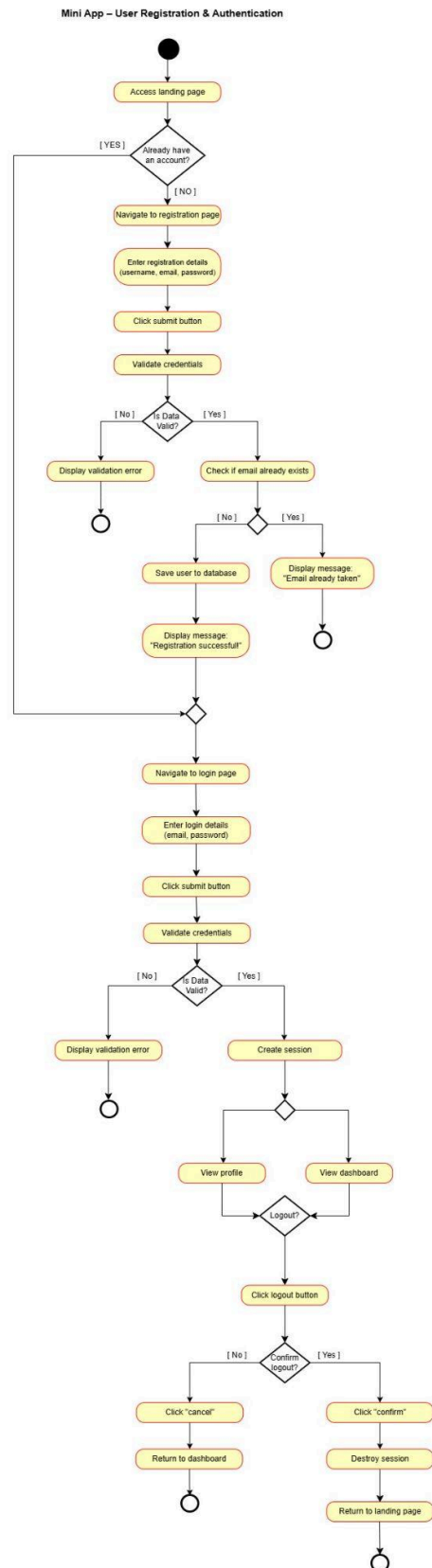
Mini App – User Registration & Authentication



### 5.2. Use Case Diagram

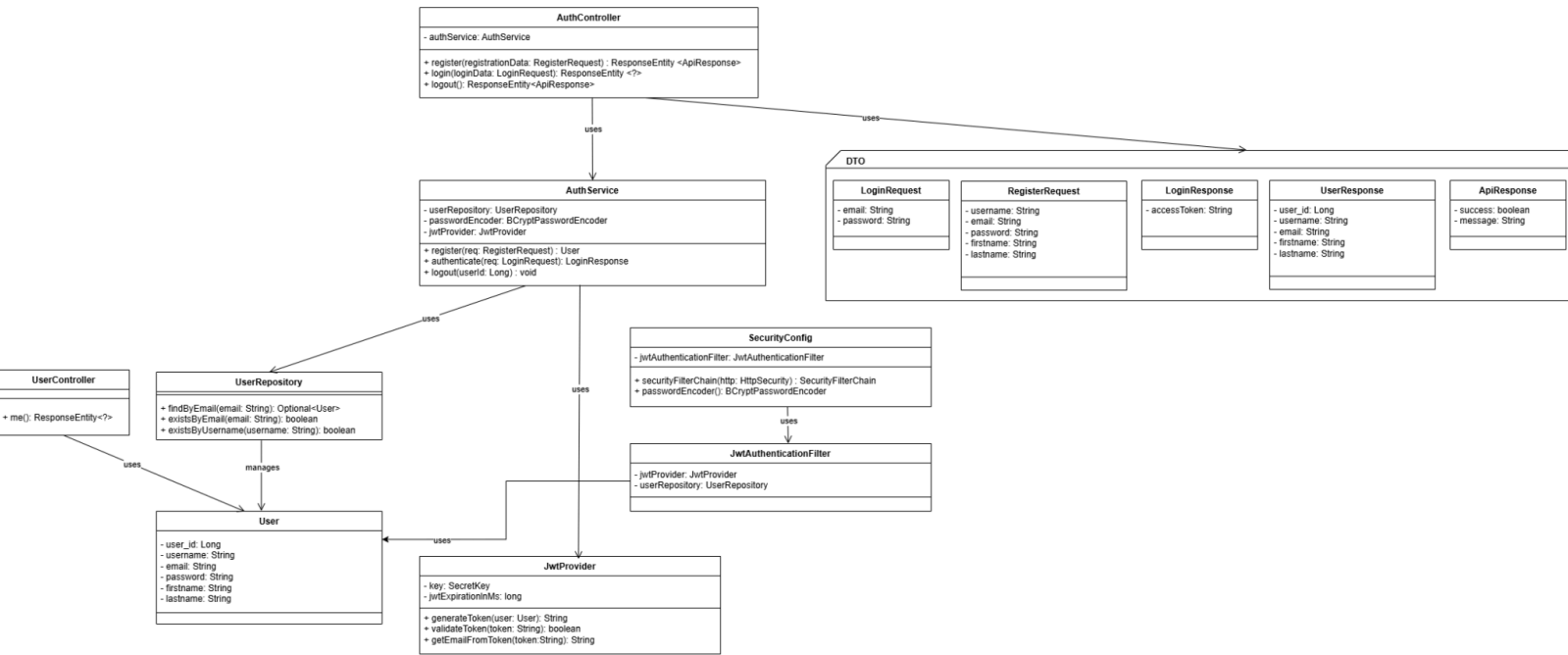


### 5.3. Activity Diagram

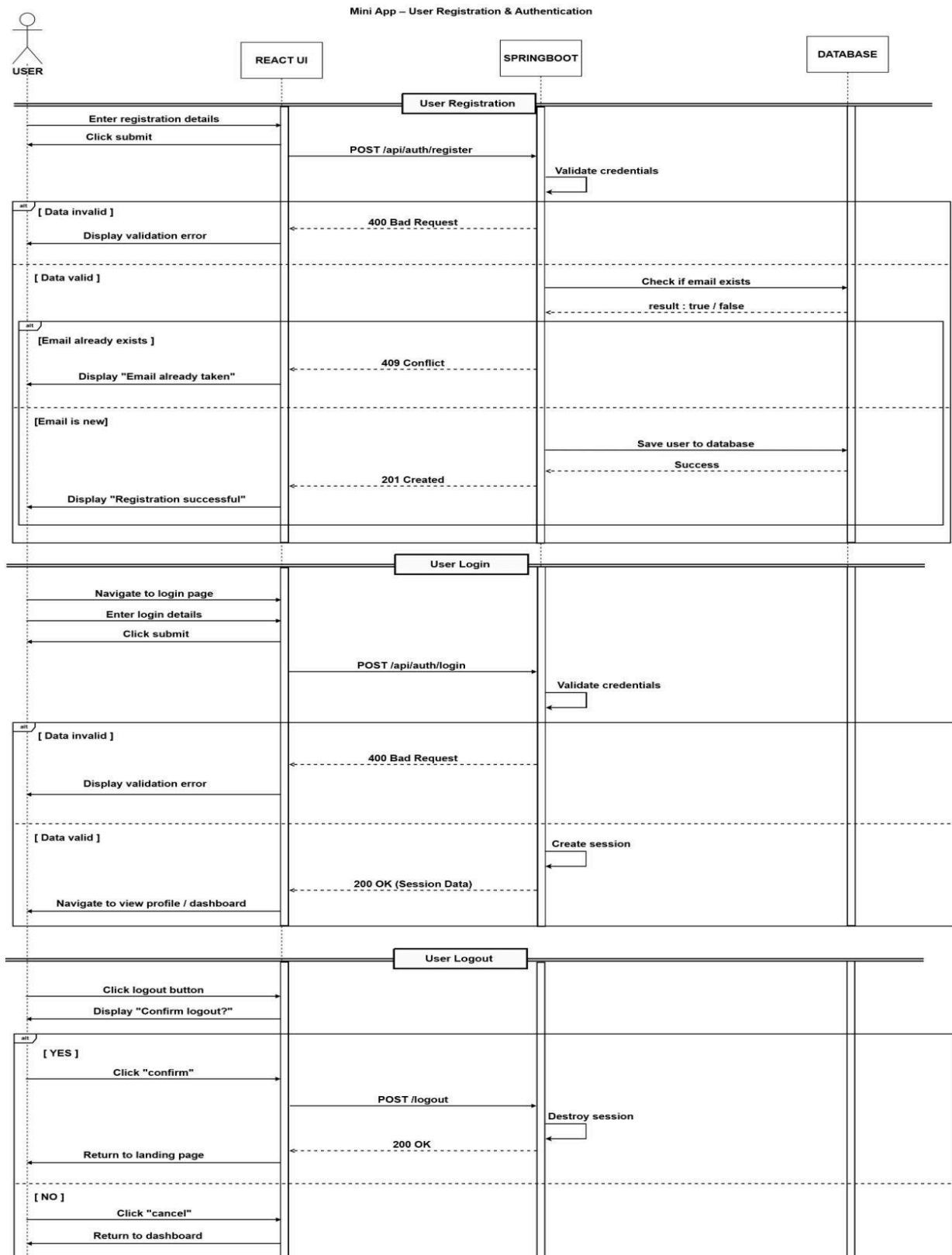


5.4. Class Diagram

Mini App – User Registration & Authentication

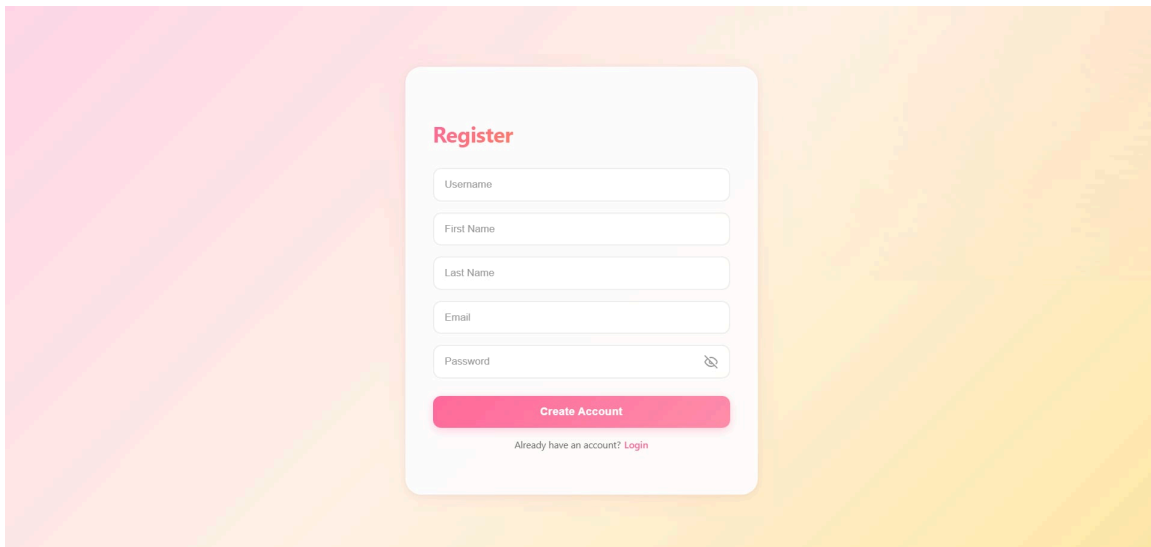
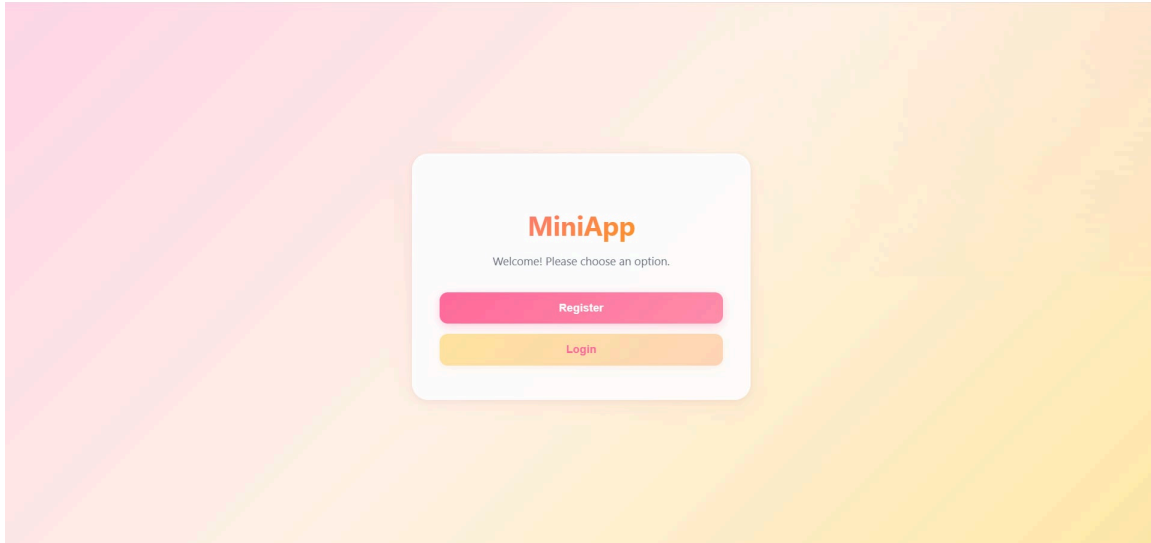


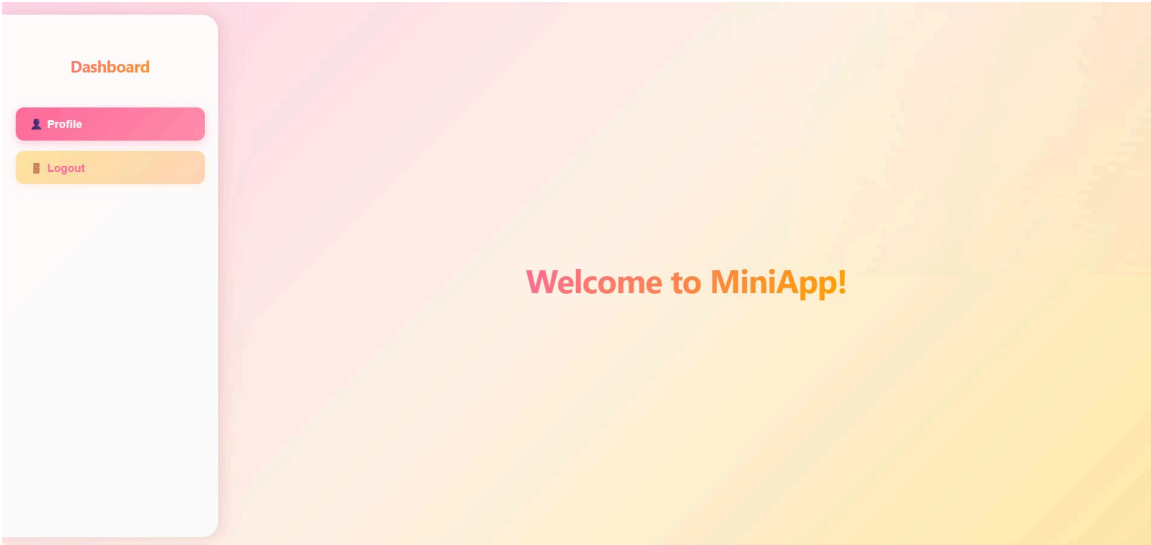
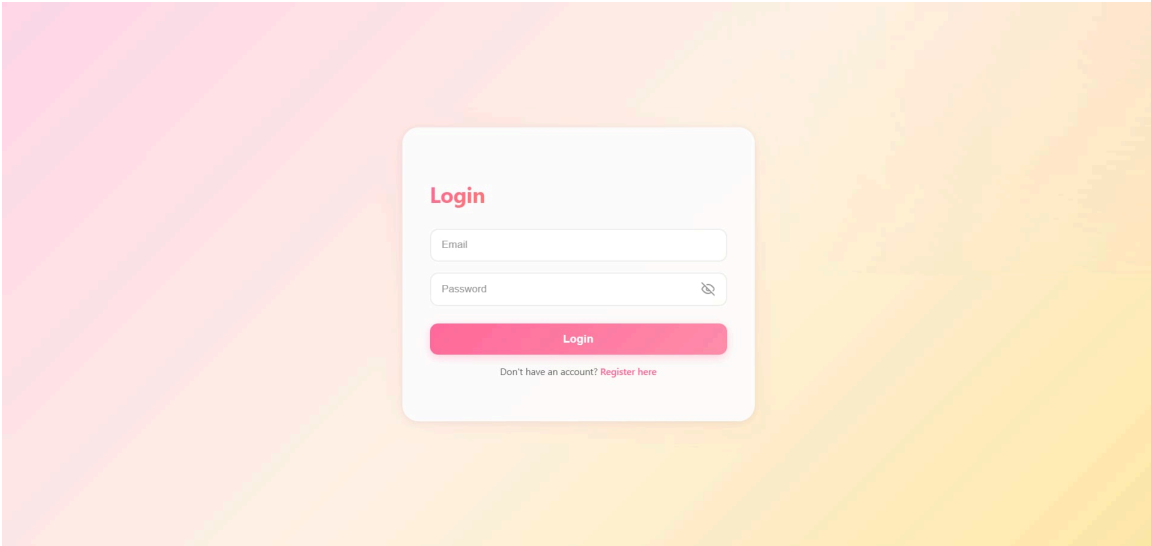
## 5.5. Sequence Diagram

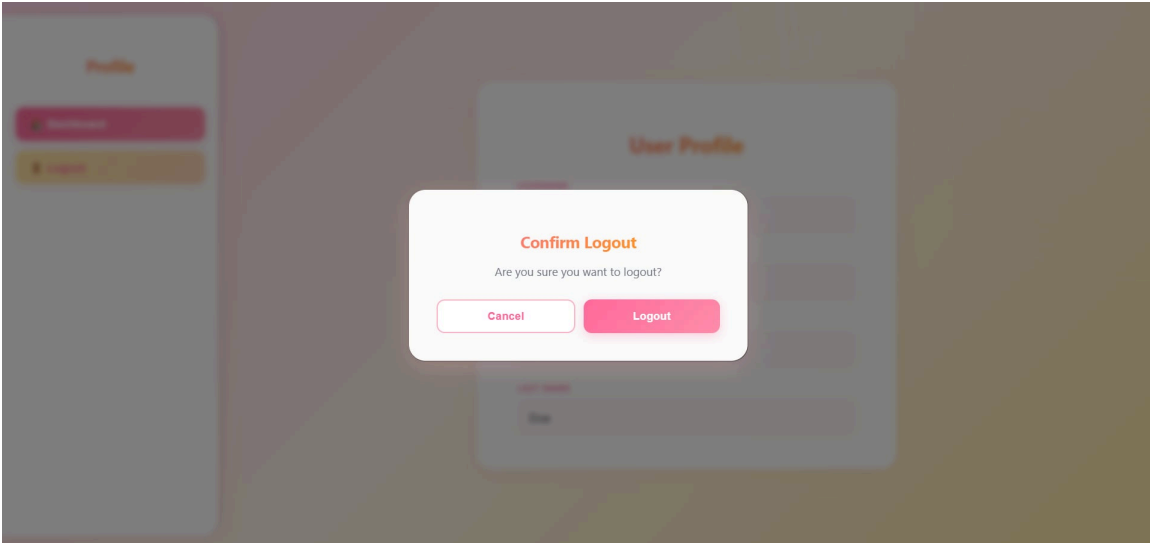
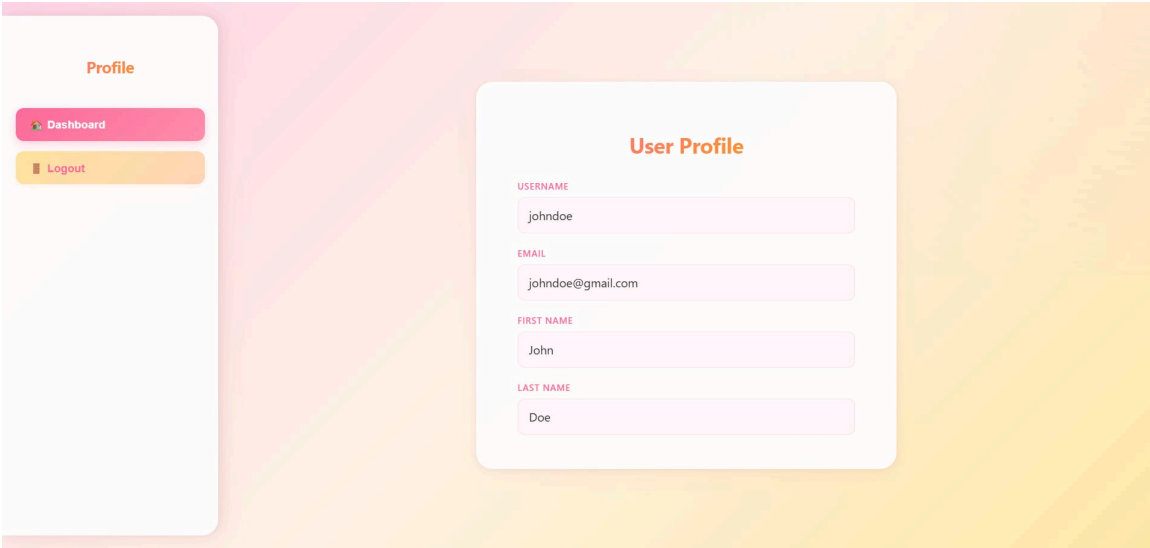


## 6. Screenshots of Web and Mobile UI

### Screenshots of Web UI







## Screenshots of Mobile UI

